

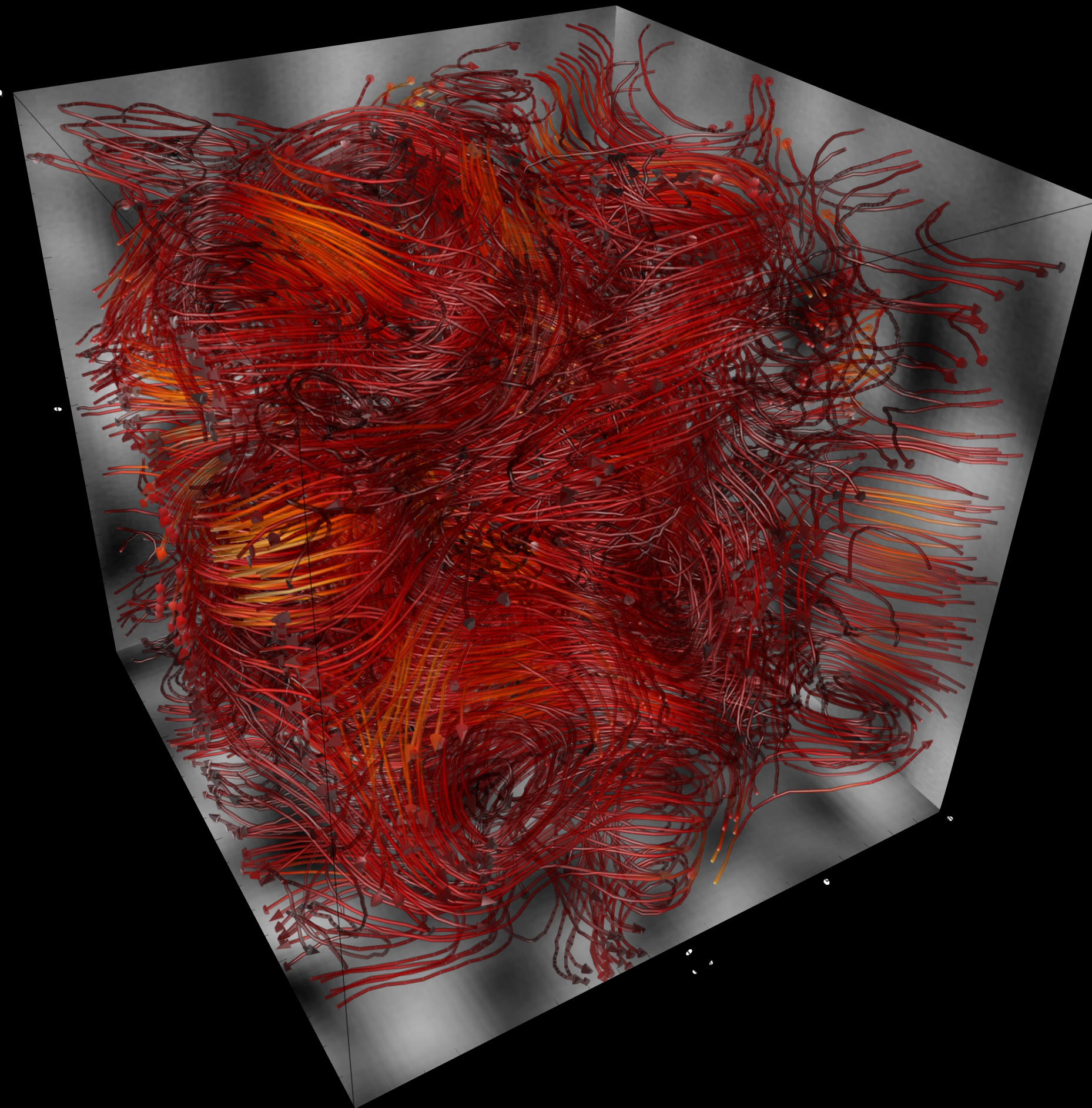
The ZPIC educational code suite

Solving kinetic plasma scenarios using

Particle-in-Cell simulations

B. Malaca and the ZPIC team

Why do it numerically?



Extreme scenarios make analytical treatment impossible

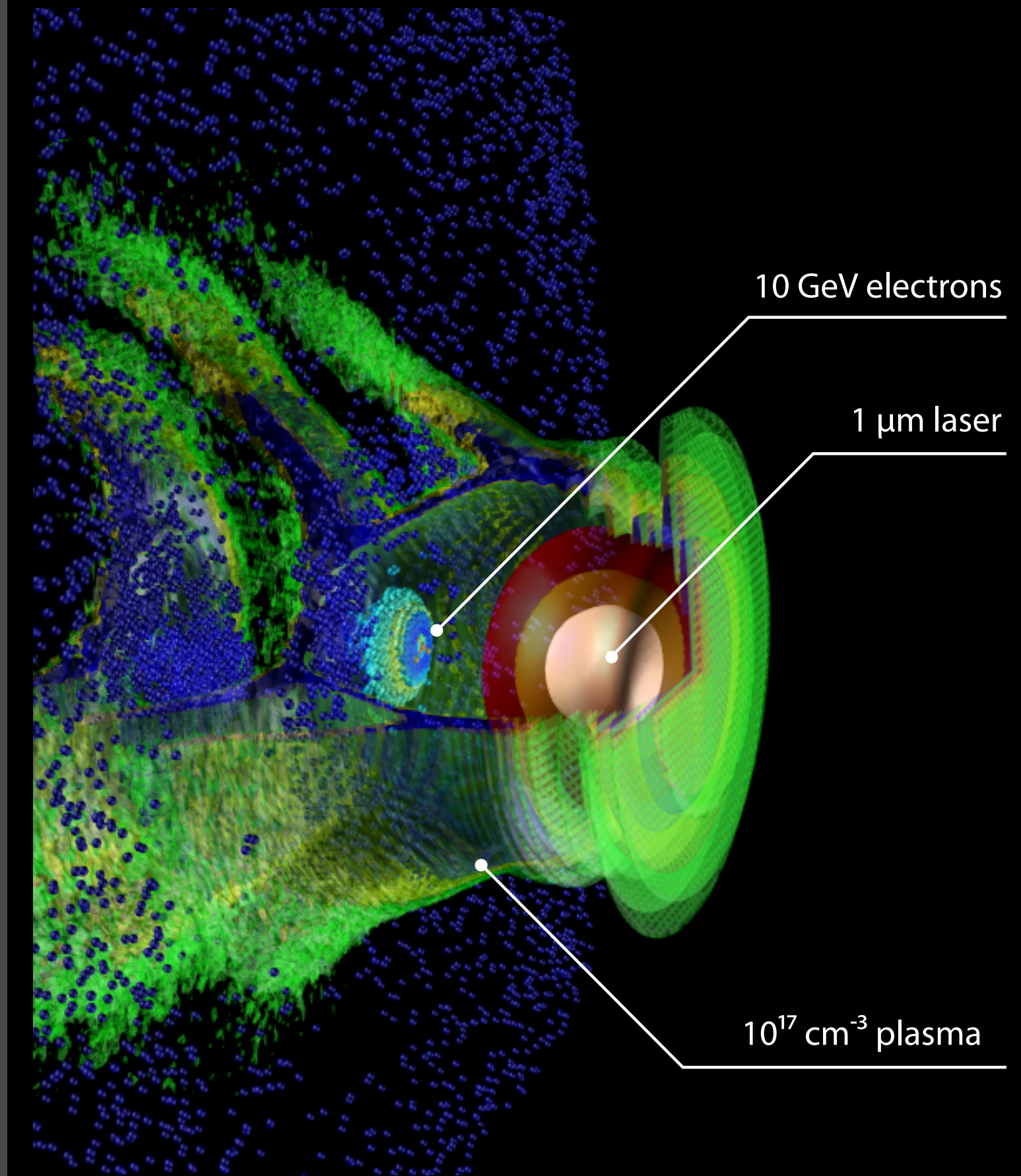
- Strong non-linearities
- Kinetic (non-equilibrium) processes
- Key tools for modelling these scenarios are simulation codes

Numerical Experiments play a central role in present scientific discovery

- Test out new theoretical concepts
- Detailed qualitative and quantitative predictions of complex scenarios
- Support of ongoing experimental work

Fundamental skill for future scientists

- Simulations play a critical role in all scientific fields
- Advances in computing technology continuously expand our numerical capabilities



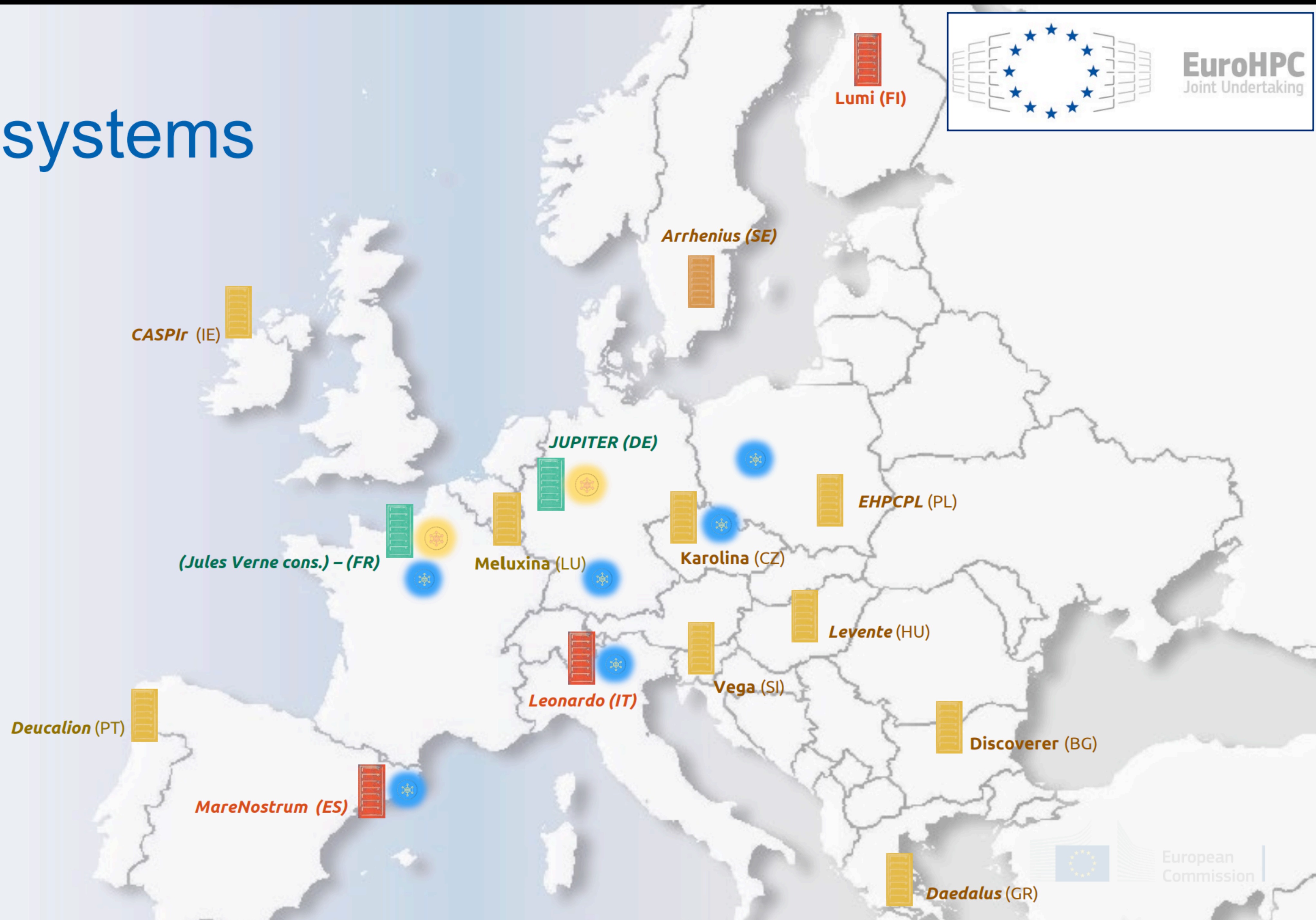
Magnetic field generation in astrophysical shocks

Laser Wakefield Acceleration

EuroHPC systems



-  **Exascale**
-  **Pre-exascale**
-  **Petascale**
-  **Qcomputer**
-  **Qsimulator**



Deucalion context (part II)

- **Deucalion is the largest computer available in Portugal.**
- **PhD thesis in Plasma physics often require tens of millions of cpu-hours.**

Deucalion hours ARM = 365 days * 24 hours * 1632 ARM nodes * 48 CPU = 686M CPU-hours

Deucalion hours x86 = 365 days * 24 hours * 500 x86 nodes * 128 CPU = 561M CPU-hours

Deucalion hours GPU = 365 days * 25 hours * 33 GPU nodes * 4 GPU = 1.15M GPU-hours

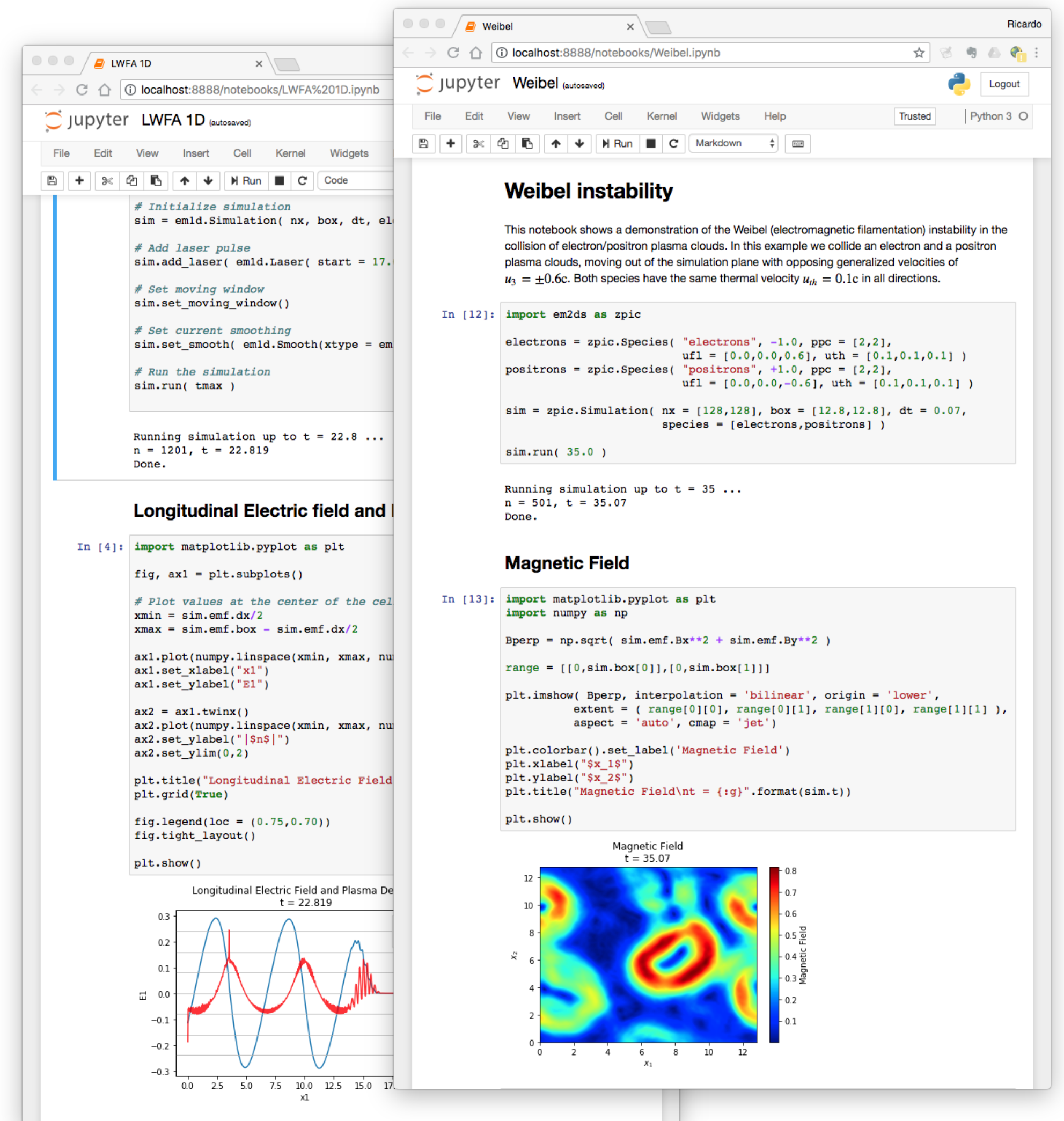
Deucalion is the only supercomputer in Portugal where you can run all the state-of-the-art plasma physics simulations that sustain the activities of a plasma physics group

And you have all access to it! As of today, this is the only course giving students access to the machine

The ZPIC educational code suite

- **ZPIC code suite**
 - Open-source PIC code suit for plasma physics education
 - Fully relativistic 1D and 2D EM-PIC algorithm
 - Eletrostatic 1D/2D PIC algorithm
- **Requirements**
 - No external dependencies for base code, requires only C99 compiler
 - Python interface included
- **Jupyter Notebooks**
 - Includes set of Python notebooks with example problems
 - Detailed explanations of code use and physics
- **Also available through Docker**

zpic@edu

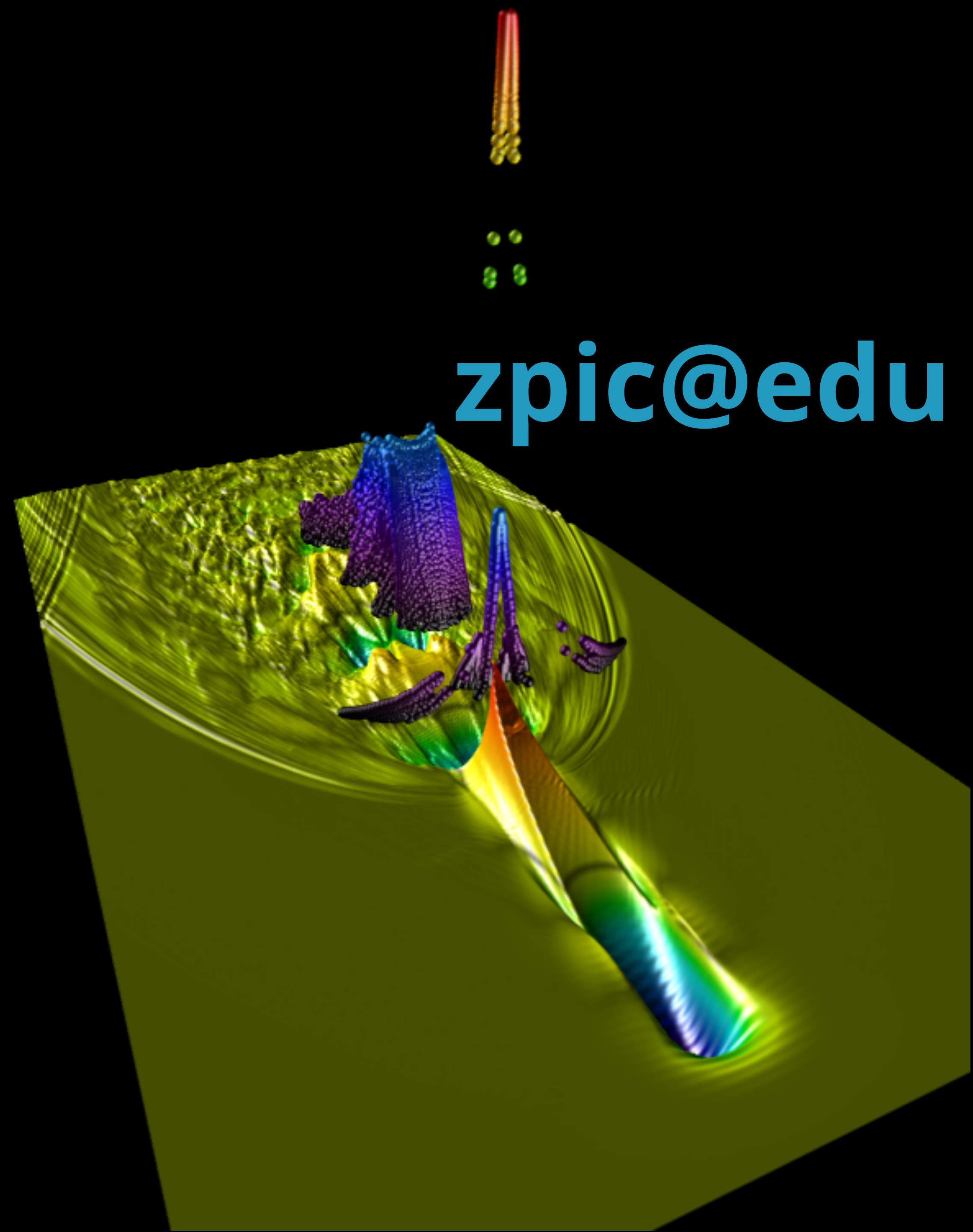


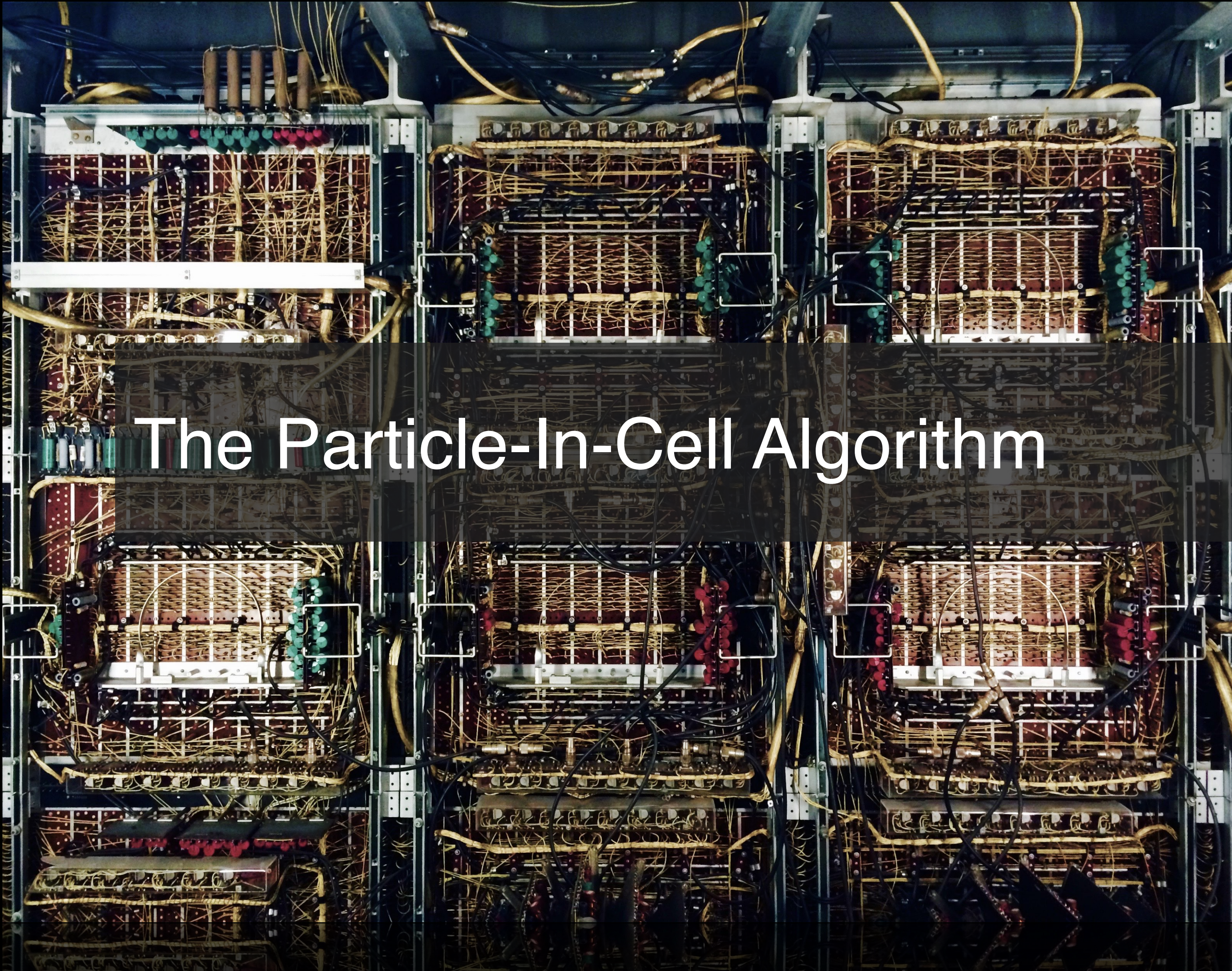
Come find us on GitHub
github.com/ricardo-fonseca/zpic

Outline

- **Review of the Particle-In Cell Algorithm**
 - Plasma simulation using particles
 - The Particle-In-Cell algorithm
 - Units and Normalization
 - Time-step considerations
- **Running ZPIC on your Computer**
 - Compiling from source

zpic@edu





The Particle-In-Cell Algorithm

UNIVAC 1 - 1951

Internal view

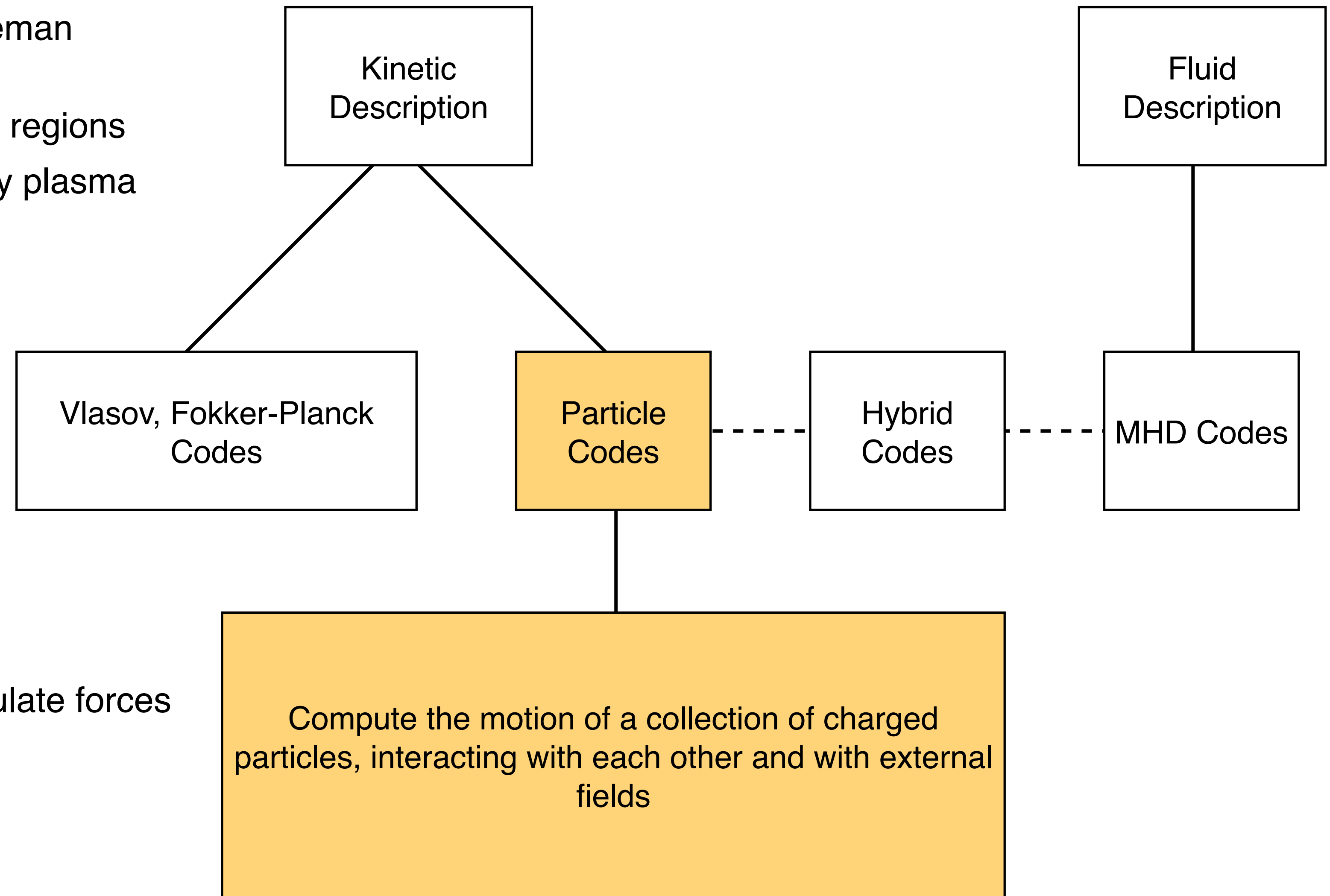
Overview of Plasma Simulation Algorithms

- **Plasma Simulations Using Particles**

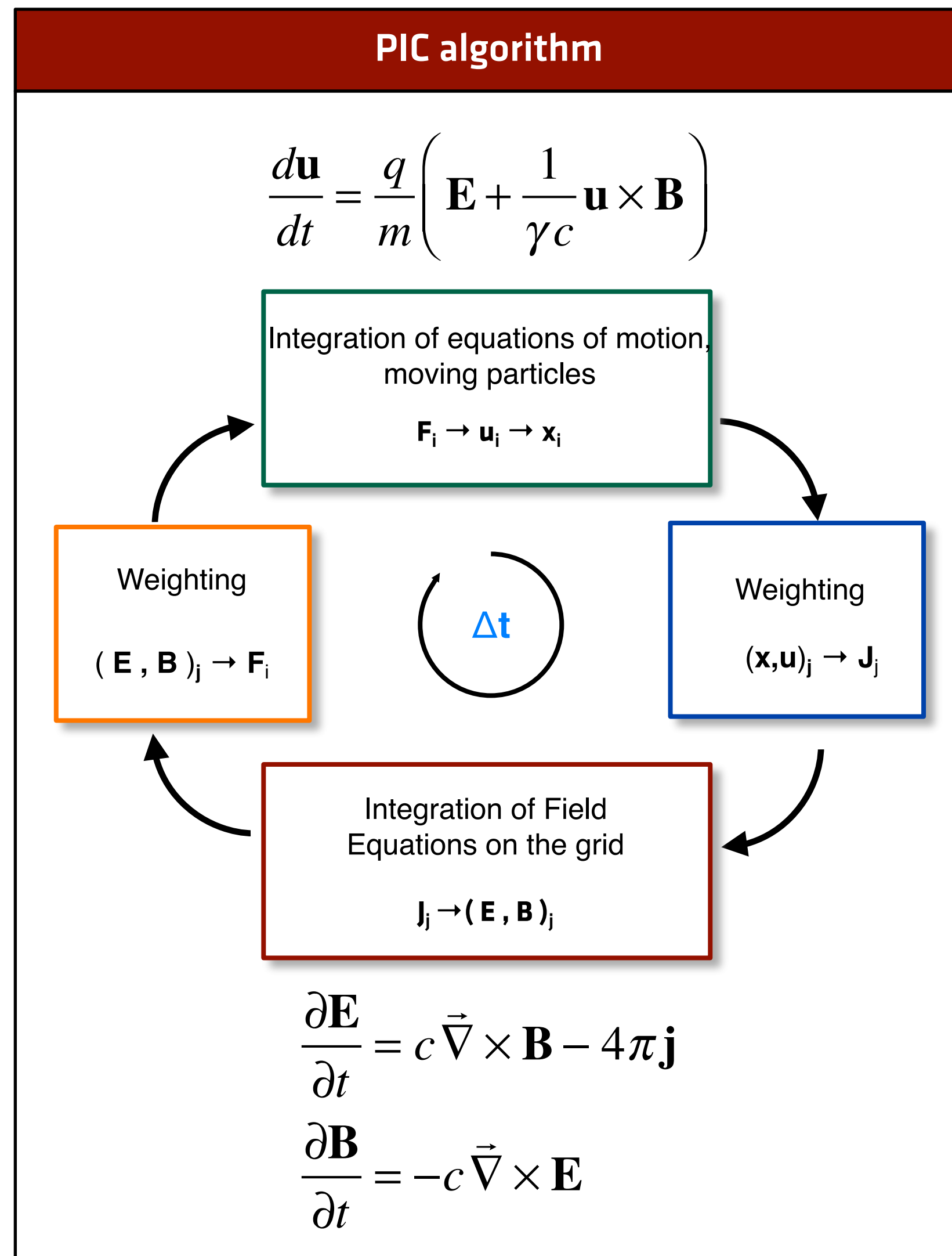
- Pioneered by John Dawson and Oscar Buneman circa 1960
- Use macro particles to simulate large spatial regions
 - 1 simulation particle corresponds to many plasma particles
- Particle-Particle simulations
 - Computations go with $O(N_p^2)$
 - Computationally very demanding

- **Particle-In-Cell algorithms**

- Interact particles through fields
- Discretize fields on grids
- Interpolate fields at particle positions to calculate forces
- Deposit particle charge/current on a grid
- Particle-Mesh algorithm
 - Computations go with $O(N_p)$
 - Still computationally heavy but much more tractable



The particle-in-cell (PIC) Algorithm



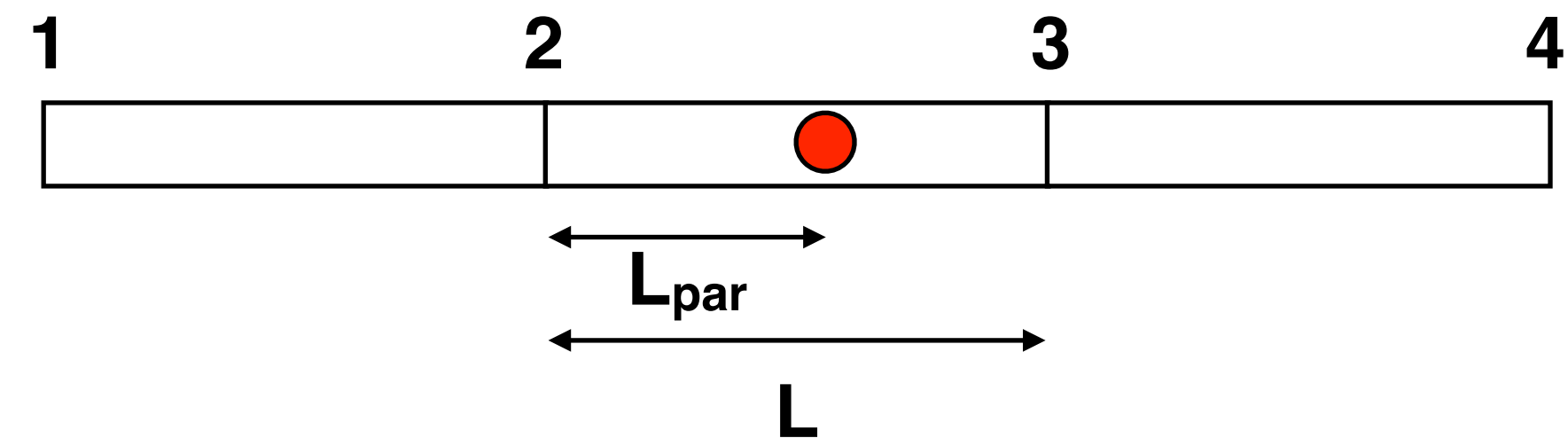
- **Fully Relativistic, Electromagnetic Particle-In-Cell algorithm**
 - Discretize Electric and Magnetic fields on a grid
 - Cell size must resolve shortest relevant lengths in the simulation
 - Typically the laser wavelength or the plasma skin depth
 - Represent plasma particles with simulation macro-particles
 - Free to move in entire $nD-3V$ phasespace
 - Each macro-particle represents several plasma particles
 - Must have enough particles per cell to properly resolve velocity distributions
- **Fields and particles don't exist in the same simulation topology**
 - Field quantities are limited to grid points
 - Field interpolation connects fields $\rightarrow$ particles
 - Current deposition connects particles $\rightarrow$ fields
- **Four major steps**
 - Field interpolation (`interpolate_fld: particles.h`)
 - Particle advance (`spec_advance: particles.h`)
 - Current deposition (`dep_current_zamb: particles.h`)
 - Field advance (`emf_advance: emf.h`)

Interpolating the fields

- **Particles are free to move to any position**
 - Field are discretized on a grid
 - Field values at particle positions are required to advance particle momenta
- **Interpolate fields at particle positions**
 - ZPIC uses linear interpolation
 - In 2D this can be viewed as area weighting
 - The interpolating scheme must be consistent with charge / current deposition
- **Momentum conserving algorithm**
 - Avoids self-forces
 - $dp/dt = 0$ for single particle

particles.h

`interpolate_fld`



$$E_{part} = \frac{L_{par}E_3 + (L - L_{par})E_2}{L}$$

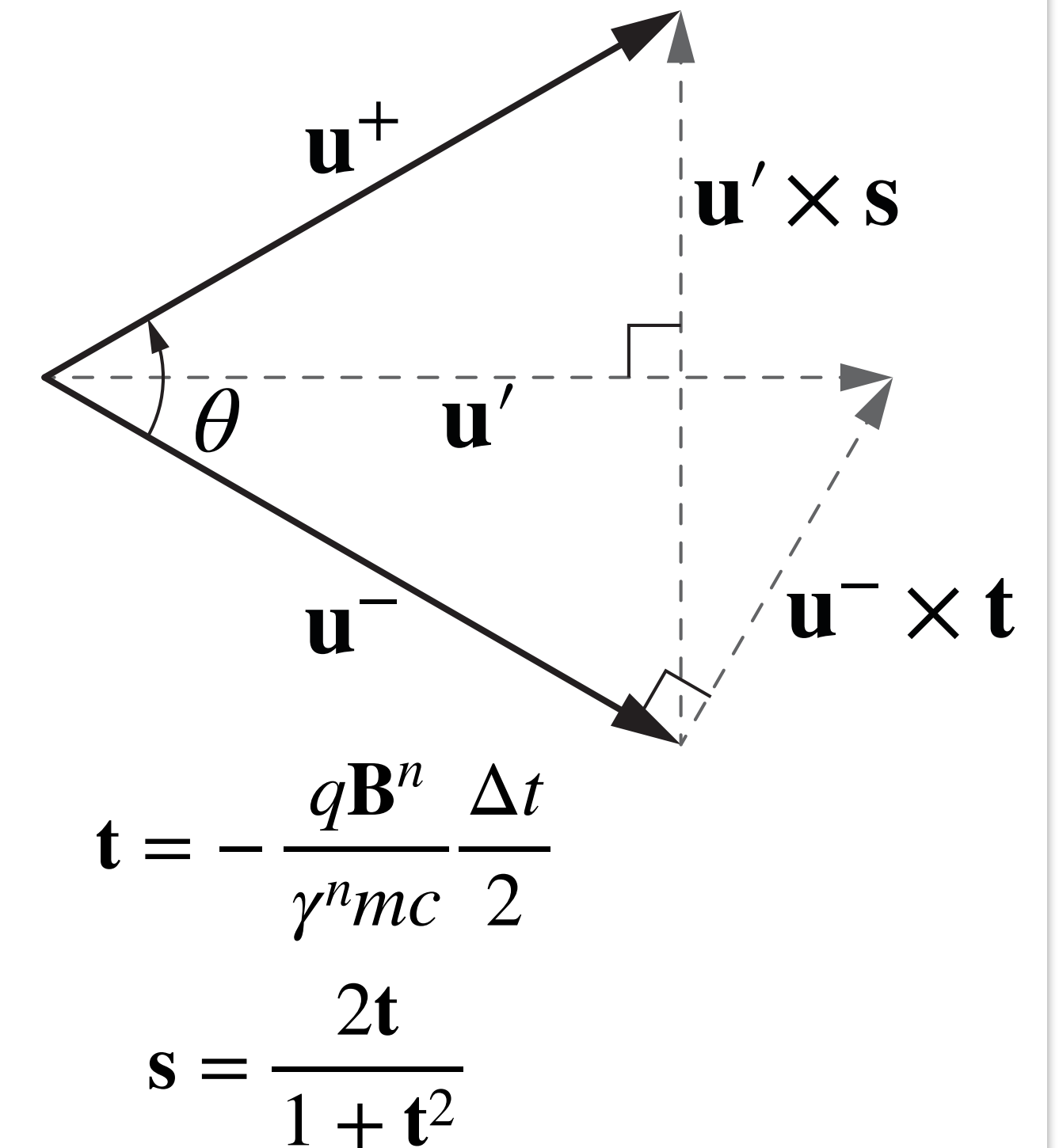
Pushing the particles

particles.h spec_advance

- **Advance generalized velocity and position of individual particles**
 - ZPIC is a fully relativistic code so we work with $\mathbf{u} = \gamma \boldsymbol{\beta}$ instead $\mathbf{v}$.
 - We use a leap-frog scheme to integrate particle motion:
 - Positions ($\mathbf{x}$) are defined at integral time-steps t^n
 - Velocities ($\mathbf{u}$) are defined at half time-steps $t^{n+1/2}$
 - Second-order accuracy in time
- **Velocities are integrated using a relativistic Boris pusher**
 - Separate $\mathbf{E}$ and $\mathbf{B}$ contributions
 - Accelerate with $\frac{1}{2}$ electric impulse
 - Full magnetic field rotation
 - Add remaining $\frac{1}{2}$ electric impulse
 - Fully relativistic, second order time accurate
 - By construction, no work from $\mathbf{B}$ field
- **Position advance is straightforward**
 - ZPIC stores cell index and position inside cell

Advance momenta

$$\begin{aligned} \text{i.} \quad & \mathbf{u}^- = \mathbf{u}^{n-1/2} + \frac{q\mathbf{E}^n}{m} \frac{\Delta t}{2} \\ \text{ii.} \quad & \mathbf{u}' = \mathbf{u}^- + \mathbf{u}^- \times \mathbf{t} \\ \text{iii.} \quad & \mathbf{u}^+ = \mathbf{u}^- + \mathbf{u}' \times \mathbf{s} \\ \text{iv.} \quad & \mathbf{u}^{n+1/2} = \mathbf{u}^+ + \frac{q\mathbf{E}^n}{m} \frac{\Delta t}{2} \end{aligned}$$



Advance positions

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \frac{\mathbf{u}^{n+1/2}}{\gamma^{n+1/2}} \Delta t$$

Depositing the current

particles.h

dep_current_zamb

- **Connects particle motion to field equations**

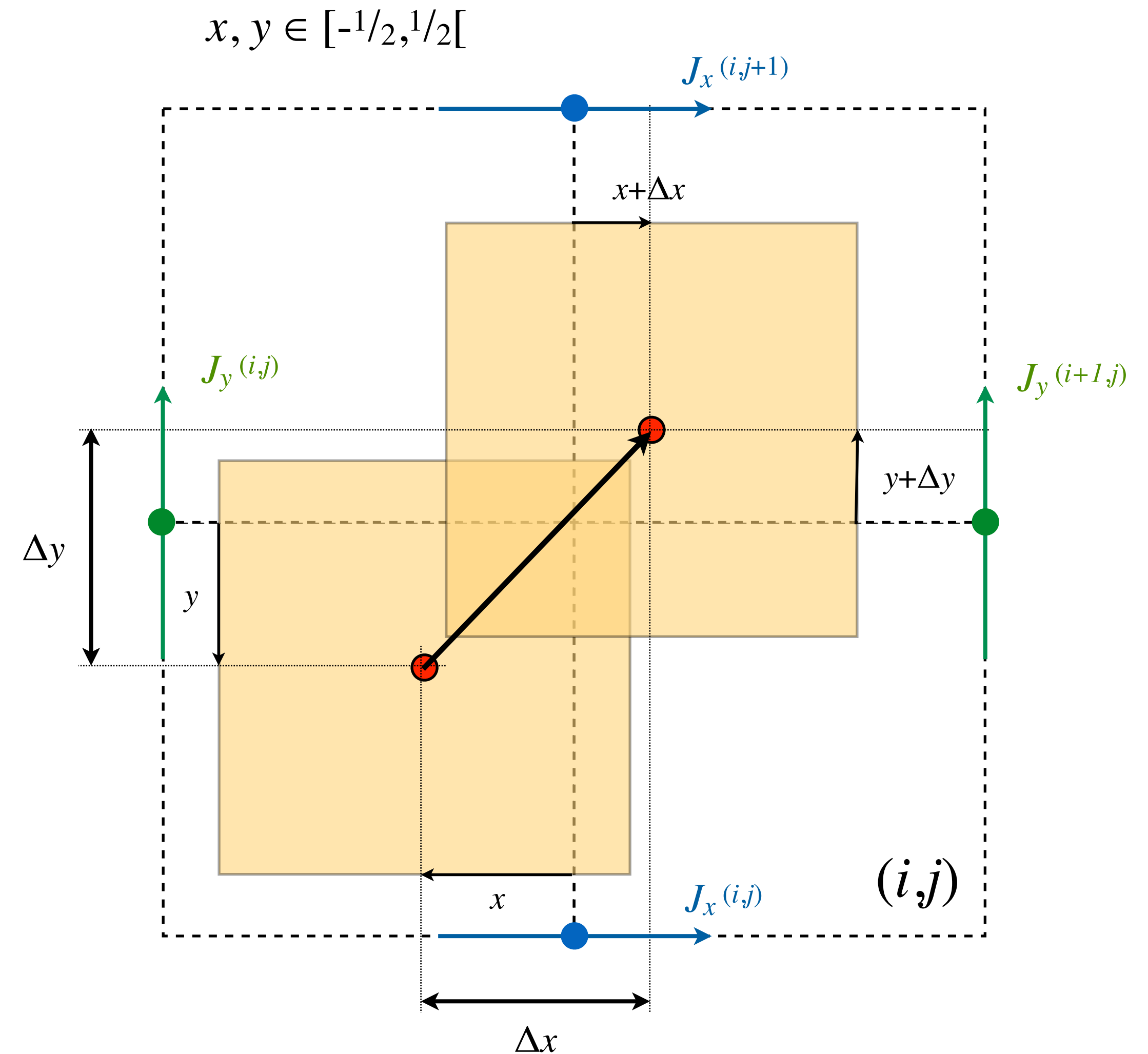
- Current deposition must satisfy continuity equation:

$$\frac{\partial \rho}{\partial t} = - \nabla' \cdot \mathbf{j}$$

- The operator ∇' corresponds to the finite difference approximation
- Simply depositing $\rho \mathbf{v}$ does not conserve charge
- Critical to guarantee the solutions to Maxwell's equations are self-consistent

- **Exact charge conserving current deposition scheme**

- Developed by Villaseñor and Buneman for linear interpolation
- Looks at particle motion, not velocity
- Limited to motion inside single cell
 - If particles cross cell boundary, motion is split into segments that don't cross boundaries



Villasenor & Buneman Comp. Phys. Comm. 69 (1992) 306-316

Advancing the EM fields

emf.h

emf_advance

- EM Fields are advanced in time using Maxwell's equations using the deposited current as source terms

- Rearrange Ampère's and Faraday's laws:

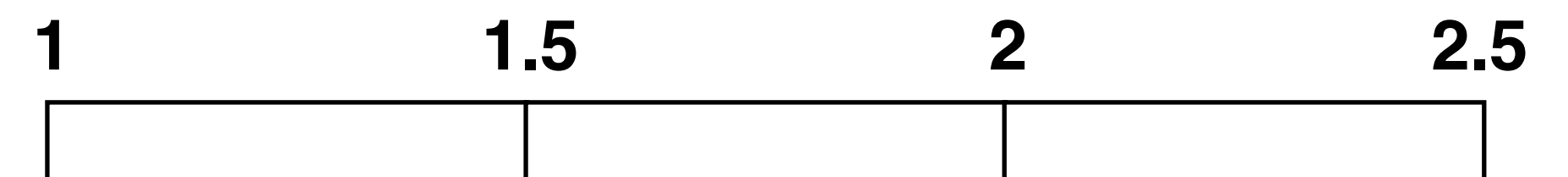
$$\frac{\partial \mathbf{E}}{\partial t} = \nabla' \times \mathbf{B} - \mathbf{j}$$

$$\frac{\partial \mathbf{B}}{\partial t} = -\nabla' \times \mathbf{E}$$

- Discretize temporal and spatial derivatives using finite differences
- Careful time and spacial centering of quantities leads to 2nd order accuracy
 - ZPIC uses the Finite Difference Time Domain (FDTD) algorithm
 - Fields are staggered in time for 2nd order accuracy
 - $\mathbf{E}$ is defined at times t^n
 - $\mathbf{B}$ and $\mathbf{j}$ are defined at times $t^{n+1/2}$
 - $\mathbf{B}$ is later time centered for use in particle advance
 - And also in space:
 - Spatial derivatives are also 2nd order accurate

```
// Canonical implementation
for (int i=-1; i<=emf->nx; i++) {
    // B[ i ].x += 0; // Bx does not evolve in 1D
    B[ i ].y += ( dt_dx * ( E[i+1].z - E[ i ].z) );
    B[ i ].z += ( - dt_dx * ( E[i+1].y - E[ i ].y) );
}
```

```
// Canonical implementation
for (int i = 0; i <= nx+1; i++) {
    E[i].x += ( - dt * J[i].x );
    E[i].y += ( - dt_dx * ( B[i].z - B[i-1].z) - dt * J[i].y );
    E[i].z += ( + dt_dx * ( B[i].y - B[i-1].y) - dt * J[i].z );
}
```



$E_y, E_z, J_y, J_z^{(1)}$
 $B_y, B_z, J_x, E_x^{(1)}$
 $E_y, E_z, J_y, J_z^{(2)}$
 $B_y, B_z, J_x, E_x^{(2)}$

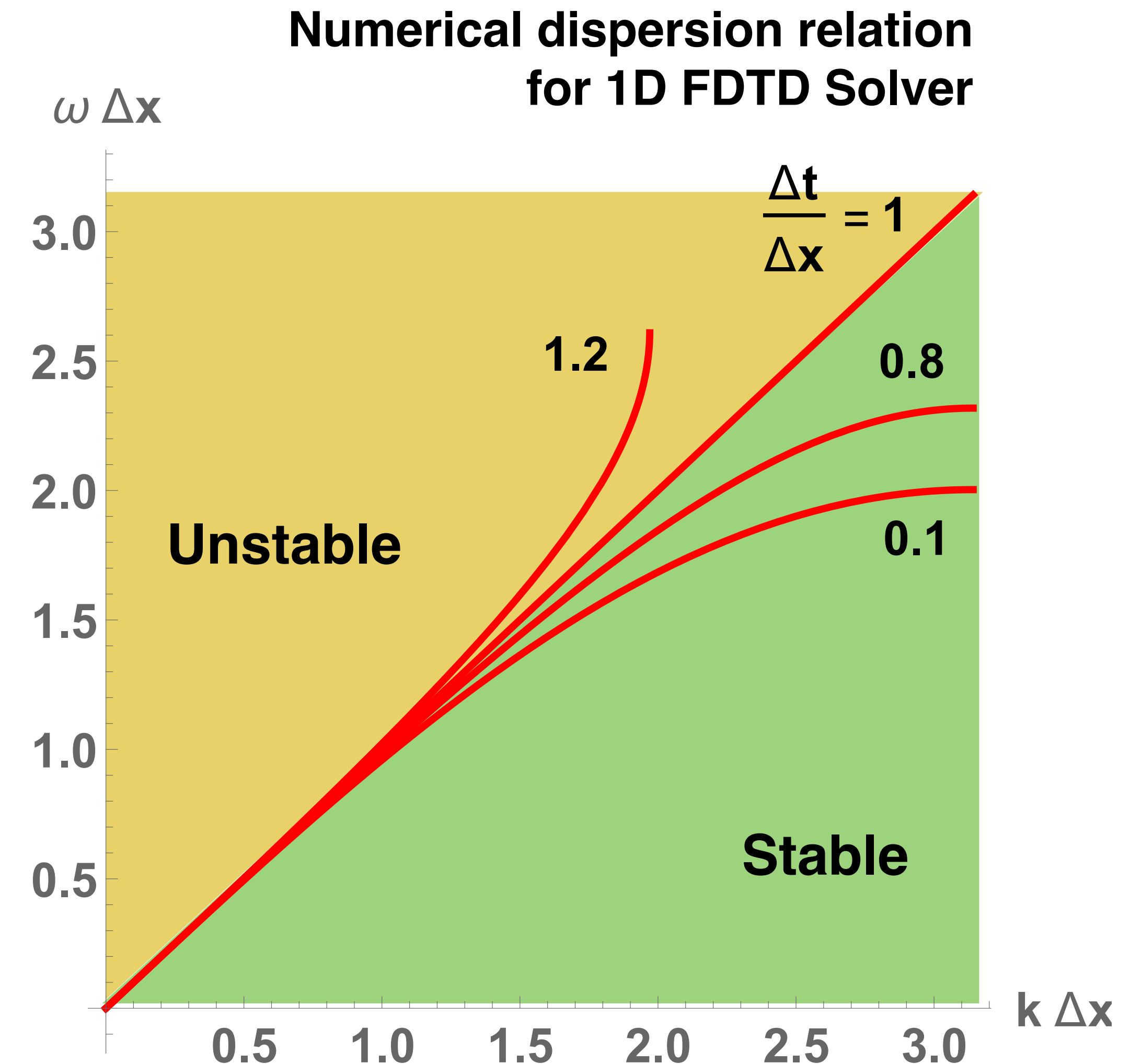
Choice of time-step

- Choice of time-step is dominated by the FDTD solver (in sim. units):

1D $\Delta t \leq \Delta x$

2D $\Delta t \leq (\Delta x^{-2} + \Delta y^{-2})^{-\frac{1}{2}}$

- If time step is larger than Courant condition the field solver becomes unstable
- If time step is much smaller than courant condition for large k , v_{ph} drops as low as $2/\pi = 0.637$ c
- Relativistic particles may have $v > v_0$
 - Numerical Cherenkov



Units and Normalization in ZPIC

- **Careful choice of units and normalization is critical**
 - Avoids multiplication by several constants (e.g. m_e , e and c) improving performance and numerical accuracy.
 - By expressing the simulation quantities in terms of fundamental plasma quantities the results are general and not bound to some specific units we may choose
- **Units and normalization in ZPIC**
 - The frequencies are normalized to a normalization frequency, ω_n . Time is normalized to ω_n^{-1} .
 - Proper velocities are normalized to the speed of light, c . Space is normalized to c/ω_n .
 - Charge and mass are normalized to the absolute electron charge, e , and the electron mass, m_e . The fields are then normalized appropriately.
 - The density is normalized to ω_n^2 (the normalization frequency squared). So if the density is 1 at a given location, then the normalization frequency is the plasma frequency at that location.

zpic units

$$\mathbf{x}' = \frac{\omega_n}{c} \mathbf{x}$$

$$\mathbf{p}' = \frac{\mathbf{p}}{m_{sp}c} = \frac{\gamma \mathbf{v}}{c} = \frac{\mathbf{u}}{c}$$

$$\mathbf{E}' = e \frac{c/\omega_n}{m_e c^2} \mathbf{E}$$

$$\mathbf{B}' = e \frac{c/\omega_n}{m_e c^2} \mathbf{B}$$

m_{sp} is the mass of the species

zpic@edu